

L14 · Δυναμικός Προγραμματισμός I

Memoization & Σταθμισμένος Χρονοπρογραμματισμός

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Η ιδέα του δυναμικού προγραμματισμού: επικαλυπτόμενα υποπροβλήματα και memoization. Fibonacci, σταθμισμένος χρονοπρογραμματισμός διαστημάτων, και η συνταγή 4 βημάτων του DP.

1 Τι θα δούμε

Ξεκινάει το τελευταίο μεγάλο κεφάλαιο: ο **δυναμικός προγραμματισμός** (dynamic programming, DP). Είναι το πιο εξεταζόμενο κομμάτι της ύλης — και αυτό που οι φοιτητές βρίσκουν πιο «μαγικό». Στόχος αυτής της διάλεξης: να πάψει να είναι μαγικό.

Θα δούμε την ιδέα μέσα από το πιο απλό παράδειγμα (Fibonacci), μετά το πρώτο «πραγματικό» πρόβλημα (**σταθμισμένος χρονοπρογραμματισμός διαστημάτων**), και θα κωδικοποιήσουμε μια **συνταγή 4 βημάτων** που εφαρμόζεται σε **κάθε** DP πρόβλημα.

2 Τα τρία αλγοριθμικά μοντέλα

Κλειδί

- **Απληστία** — χτίσε τη λύση σταδιακά, βελτιστοποιώντας μυωπικά ένα τοπικό κριτήριο.
- **Διαίρει και κυρίευε** — διάσπασε σε **ανεξάρτητα** υποπροβλήματα, λύσε το καθένα ξεχωριστά, συνδύασε.
- **Δυναμικός προγραμματισμός** — διάσπασε σε μια σειρά από **επικαλυπτόμενα** υποπροβλήματα, και δόμησε σωστές λύσεις για όλο και μεγαλύτερα υποπροβλήματα.

Διαίσθηση

Η κρίσιμη λέξη είναι «**επικαλυπτόμενα**». Στο διαίρει-και-κυρίευε τα δύο μισά ενός πίνακα είναι **ξένα** — δεν μοιράζονται τίποτα. Στον δυναμικό προγραμματισμό τα υποπροβλήματα **ξανασυναντιούνται**: το ίδιο υποπρόβλημα χρειάζεται πολλές φορές. Αν το λύνεις ξανά κάθε φορά, πληρώνεις εκθετικά. Αν το λύνεις **μία φορά και το θυμάσαι**, πληρώνεις γραμμικά. Αυτή είναι όλη η ιδέα.

Σημείωση

Από πού το όνομα; Ο Richard Bellman θεμελίωσε τον δυναμικό προγραμματισμό τη δεκαετία του 1950. «Programming» εδώ σημαίνει **σχεδιασμός** (όπως στο «linear programming»), όχι κώδικας. Ο Bellman διάλεξε επίτηδες ένα εντυπωσιακό όνομα — κάτι που ούτε ένα μέλος του Κογκρέσου δεν θα μπορούσε να απορρίψει ως χρηματοδότηση. Διάσημοι DP αλγόριθμοι: **Viterbi, unix diff**,

3 Το παράδειγμα-εκκίνηση: Fibonacci

Η ακολουθία Fibonacci ορίζεται αναδρομικά:

$$F(n) = F(n-1) + F(n-2), \quad F(0) = 0, F(1) = 1.$$

Η απευθείας μεταφορά σε κώδικα:

```
fib(n):
  Εάν n <= 1: επίστρεψε n
  επίστρεψε fib(n-1) + fib(n-2)
```

Κομψό — και **καταστροφικό**. Η πολυπλοκότητα είναι $\approx O(2^n)$, εκθετική.

3.1 Γιατί είναι τόσο αργό; Επικάλυψη υποπροβλημάτων

Όταν υπολογίζουμε το $F(5)$ με την αφελή αναδρομή, το δέντρο κλήσεων υπολογίζει το $F(3)$ **2 φορές** και το $F(2)$ **3 φορές**. Όσο μεγαλώνει το n , το δέντρο αναδρομής **εκρήγνυται** — επειδή τα ίδια υποπροβλήματα υπολογίζονται ξανά και ξανά.

3.2 Η λύση: memoization

Κλειδί

Απομνημόνευση (memoization): κάθε φορά που υπολογίζεις έναν όρο $F(k)$, **αποθήκευσέ τον** σε έναν πίνακα. Όταν τον ξαναζητήσει η αναδρομή, **επίστρεψε την αποθηκευμένη τιμή** αντί να ξαναξεκινήσεις τον υπολογισμό.

```
fib2(n):
  Εάν n <= 1: επίστρεψε n
  A[0] <- 0 ; A[1] <- 1
  Για i = 2 έως n:
    A[i] <- A[i-1] + A[i-2]
  επίστρεψε A[n]
```

Κάθε $F(k)$ υπολογίζεται **ακριβώς μία φορά**. Η πολυπλοκότητα πέφτει από εκθετική $O(2^n)$ σε **γραμμική** $O(n)$.

4 Σταθμισμένος Χρονοπρογραμματισμός Διαστημάτων

Τώρα το πρώτο πραγματικό πρόβλημα. Είναι το πρόβλημα του χρονοπρογραμματισμού διαστημάτων από το μάθημα των άπληστων αλγορίθμων — **με μια προσθήκη**.

Είσοδος: n αιτήματα. Το αίτημα j ξεκινά τη στιγμή s_j , τελειώνει τη στιγμή f_j , και έχει **βαρύτητα** v_j . Δύο αιτήματα είναι **συμβατά** αν δεν επικαλύπτονται.

Στόχος: υποσύνολο S ανά δύο συμβατών αιτημάτων με **μέγιστο άθροισμα βαρυτήτων** $\sum_{i \in S} v_i$.

Προσοχή

Ο άπληστος αλγόριθμος αποτυγχάνει θεαματικά. Στους άπληστους αλγορίθμους, με όλες τις βαρύτητες ίσες με 1, το «μικρότερος χρόνος λήξης» έδινε βέλτιστη λύση. Με αυθαίρετες βαρύτητες όμως, μια εργασία που τελειώνει νωρίς αλλά αξίζει 1 μπορεί να μπλοκάρει μία που αξίζει 100. Κανένα απλό κριτήριο σειράς δεν αρκεί — χρειαζόμαστε DP.

4.1 Η προετοιμασία: ταξινόμηση και $p(j)$

Διατάσσουμε τα αιτήματα κατά **χρόνο λήξης**: $f_1 \leq f_2 \leq \dots \leq f_n$. Ορίζουμε:

Κλειδί

$p(j)$ = ο **μεγαλύτερος** δείκτης $i < j$ τέτοιος ώστε το αίτημα i να είναι **συμβατό** με το j (δηλαδή $f_i \leq s_j$). Αν κανένα προηγούμενο αίτημα δεν είναι συμβατό, $p(j) = 0$.

Διαισθητικά: «αν διαλέξω το j , ποιο είναι το **τελευταίο** προηγούμενο αίτημα που μπορώ να συνδυάσω μαζί του;»

4.2 Η αναδρομή — δυαδική επιλογή

Ορίζουμε $OPT(j)$ = η τιμή της βέλτιστης λύσης για τα αιτήματα $\{1, 2, \dots, j\}$. Σκεφτόμαστε το αίτημα j : **μέσα ή έξω**;

- **Το j είναι ΜΕΣΑ στη βέλτιστη λύση.** Τότε αποκλείονται όλα τα ασύμβατα με το j — δηλαδή τα $\{p(j) + 1, \dots, j - 1\}$. Ό,τι μένει είναι η βέλτιστη λύση για τα $\{1, \dots, p(j)\}$. Συνεισφορά: $v_j + OPT(p(j))$.
- **Το j είναι ΕΞΩ.** Τότε η βέλτιστη λύση είναι ακριβώς η βέλτιστη για τα $\{1, \dots, j - 1\}$. Συνεισφορά: $OPT(j - 1)$.

Δεν ξέρουμε ποια περίπτωση ισχύει — οπότε παίρνουμε τη **μεγαλύτερη**:

Κλειδί

$$OPT(j) = \begin{cases} 0 & j = 0 \\ \max\{v_j + OPT(p(j)), OPT(j - 1)\} & j \geq 1 \end{cases}$$

4.3 Από την ωμή βία στο memoization

Η απευθείας αναδρομή $ComputeOPT(j)$ είναι **εκθετική** — ακριβώς όπως ο αφελής Fibonacci, το πλήθος αναδρομικών κλήσεων μεγαλώνει σαν $T(n) = T(n - 1) + T(n - 2)$. Η θεραπεία είναι η ίδια: **απομνημόνευση**.

Αρχικοποίησε $M[1..n] = \text{κενό}$; $M[0] = 0$

M-ComputeOPT(j):

Εάν $M[j] = \text{κενό}$:

$M[j] \leftarrow \max\{v_j + M-ComputeOPT(p(j)), M-ComputeOPT(j - 1)\}$

επίστρεψε $M[j]$

Κλειδί

Χρόνος εκτέλεσης: $O(n \log n)$.

- Ταξινόμηση κατά χρόνο λήξης: $O(n \log n)$.
- Υπολογισμός όλων των $p(j)$: $O(n)$ μετά από ταξινόμηση και κατά χρόνο έναρξης.
- M-ComputeOPT: κάθε κλήση είναι $O(1)$ · κάθε νέα συμπλήρωση μιας θέσης $M[j]$ κάνει το πολύ 2 αναδρομικές κλήσεις. Μέτρο προόδου Φ = πλήθος μη-κενών θέσεων του $M[\cdot]$ · ξεκινά 0, φτάνει $\leq n$, και κάθε «νέα» κλήση το αυξάνει κατά 1. Άρα συνολικά $O(n)$ κλήσεις.

Ισοδύναμα, **bottom-up** — γεμίζουμε τον πίνακα με αύξουσα σειρά, χωρίς αναδρομή:

```
M[0] <- 0
Για j = 1 έως n:
    M[j] <- max{ v_j + M[p(j)], M[j-1] }
```

4.4 Εύρεση της λύσης

Ο αλγόριθμος υπολογίζει τη **μέγιστη τιμή**. Για να βρούμε **ποια αιτήματα** την πετυχαίνουν, κάνουμε ένα πέρασμα προς τα πίσω, ρωτώντας σε κάθε j «ποια από τις δύο περιπτώσεις κέρδισε;»

```
Βρες-Λύση(j):
    Εάν j = 0: μην εκτυπώσεις τίποτα
    Αλλιώς εάν v_j + M[p(j)] > M[j-1]: // το j ήταν ΜΕΣΑ
        εκτύπωσε j
        Βρες-Λύση(p(j))
    Αλλιώς: // το j ήταν ΕΞΩ
        Βρες-Λύση(j-1)
```

Το πολύ n κλήσεις, άρα $O(n)$.

5 Η συνταγή του δυναμικού προγραμματισμού

Κάθε πρόβλημα DP — όσα θα δούμε στα επόμενα μαθήματα — λύνεται με τα **ίδια 4 βήματα**:

Κλειδί

1. **Χαρακτηρισμός της δομής** του προβλήματος — όρισε τι είναι ένα «υποπρόβλημα» (εδώ: «τα πρώτα j αιτήματα»).
2. **Αναδρομικός ορισμός** της τιμής της βέλτιστης λύσης — η εξίσωση OPT($\cdot$).
3. **Υπολογισμός** της τιμής της βέλτιστης λύσης — γέμισε τον πίνακα (memoization ή bottom-up).
4. **Κατασκευή** της βέλτιστης λύσης από την αποθηκευμένη πληροφορία — το πέρασμα προς τα πίσω.

Διαίσθηση

Το πιο δύσκολο βήμα είναι το 1ο. Μόλις βρεις τη σωστή «παραμετροποίηση» των υποπροβλημάτων, η αναδρομή σχεδόν γράφεται μόνη της. Σε επόμενο μάθημα θα δούμε πρόβλημα (knapsack) όπου η πρώτη, «προφανής» παραμετροποίηση **δεν δουλεύει** — και χρειάζεται μια

δεύτερη μεταβλητή.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Δυναμικός προγραμματισμός** — διάσπαση σε **επικαλυπτόμενα** υποπροβλήματα· λύσε καθένα μία φορά, αποθήκευσέ το.
2. **Fibonacci** — αφελής αναδρομή $O(2^n)$ λόγω επικάλυψης· **memoization** $\rightarrow O(n)$.
3. **Σταθμισμένος χρονοπρογραμματισμός** — μέγιστη βαρύτητα συμβατών αιτημάτων. Ο άπληστος αποτυγχάνει.
4. $p(j)$ = τελευταίο συμβατό προηγούμενο αίτημα. Αναδρομή: $OPT(j) = \max\{v_j + OPT(p(j)), OPT(j-1)\}$.
5. Με memoization / bottom-up $\rightarrow O(n \log n)$ · εύρεση λύσης με πέρασμα προς τα πίσω $\rightarrow O(n)$.
6. **Η συνταγή DP (4 βήματα)**: δομή $\rightarrow$ αναδρομικός ορισμός $\rightarrow$ υπολογισμός τιμής $\rightarrow$ κατασκευή λύσης.

Σημειώσεις από το *Algorithms Class Hub* — μελετητικός οδηγός για το μάθημα «Αλγόριθμοι και Πολυπλοκότητα (Κ17)», ΕΚΠΑ.